

第2回 ハッカソン2 事前課題

学籍番号：24G1021

氏名：宇井祐真

2026 年 4 月 22 日

1 提案する部品

通信方式

2 技術的・社会的な背景

Arduino UNO R4 WiFi の CPU は arm cortex m4 である [1]. この CPU はシングルコアであるため、同時に一つの処理しか実行できない。そのため、楽器の指揮をする arduino は、楽器を操作する arduino との通信と同期を表現するための処理を同時に実行できない。しかし、楽器の指揮をする場合、同期するための通信と同期を表現するための処理を同時に行う必要がある。

3 既存のシステムとその課題

シングルコア CPU で複数の処理を行う手法として、タイマー割り込みやタスクスケジューラを用いた時分割処理がある [2]. しかし、これらは処理を順番に待つ必要があるため、通信待ちが発生すると他の処理が停止する問題がある

本システムでは、通信処理と同期表現の処理を並列に処理する必要がある。これを実現する技術に FreeRTOS がある [3]. FreeRTOS は、プリエンプティブマルチタスクと言われる、並行して複数の処理を実行させることができる OS である。しかし、FreeRTOS は、タスクの切り替えやメモリ管理にリソースを消費するため、タスクの作り方によっては、パフォーマンスが低下するリスクがある。

本システムで使用する通信は無線を想定しており、無線での通信方式には、bluetooth, WiFi, ESP NOW などがある。bluetooth は接続の手間が多く、WiFi や ESP NOW と比較すると伝送速度が劣る [4]. WiFi を使用する場合は、TCP を利用することで信頼性の高い通信が可能だが、TCP/IP のオーバーヘッドの処理が重く、CPU リソースを消費してしまう [5]. ESP NOW は、OSI 参照モデルの第2層であるデータリンク層で通信するため、上位層の処理の必要がなく、低遅

延で軽量である [6]。しかし、TCP にあるような信頼性の保証がないため、パケットロスが起こる可能性がある。

4 自身のアイデア

課題の解決策として、FreeRTOS を使用して、並列処理で通信と同期表現を実装する。FreeRTOS のリソースを確保するために、通信方式として ESP NOW を使用して、通信のオーバーヘッドなどを減らす。ESP NOW を使った通信の信頼性が保証されない問題については、低遅延で軽量であるという利点を活かして、何度も通信を繰り返すことで、多少のパケットロスは許容できるようにすることで解決する。

参考文献

- [1] Arduino UNO R4 WiFi データシート, <https://akizukidenshi.com/goodsaffix/ABX00087.pdf>, (参照 2026-04-21).
- [2] ARDUINODOCS, TaskScheduler, <https://docs.arduino.cc/libraries/taskscheduler/>, (参照 2026-04-21).
- [3] aws, FreeRTOS, <https://aws.amazon.com/jp/freertos/>, (参照 2026-04-21).
- [4] NEXTY Electronics, Bluetooth 技術コラム, https://www.nexty-ele.com/technical-column/bluetooth_01/, (参照 2026-04-21).
- [5] RFC, Transmission Control Protocol (TCP), <https://www.rfc-editor.org/rfc/rfc9293.txt>, (参照 2026-04-21).
- [6] ESPRESSIF, ESP-NOW, https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/api-reference/network/esp_now.html, (参照 2026-04-21).